

Does JAM maintain its capacity to correct shared implementation faults?

A narrow question for expert review of the JAM/ELVES security model. The aim is not to propose a replacement protocol model, but to test whether a residual failure mode has been identified or whether JAM already closes it elsewhere.

The question

Fix one invalid report and one fault family x . Some validators may be adversarial. A second group may be honest in the ordinary sense but share an implementation fault that makes this particular invalid report look valid. A third group independently detects the fault.

Core distinction. Validator honesty and implementation independence are different security resources. A validator can be honest yet fail to provide independent corrective capacity for a particular fault family.

Let A be adversarial validators, B_x otherwise-honest validators sharing fault x , and C_x validators that independently judge the report invalid, with $N = A + B_x + C_x$.

At the Gray-Paper verdict layer, the relevant arithmetic can be stated directly: if $K = \lfloor 2N/3 \rfloor + 1$, then independently correct negative judgments can construct a bad verdict against adversarial positive voting exactly when $C_x \geq K$ in the declared fault partition.

For the ELVES-style escalation model, let F be the tranche/escalation factor, $\gamma = A/N$, and $f_x = B_x/(N - A)$. Replacing the generic honest share by the fault-specific independently correct share gives the declared reproduction mean

$$\lambda_x = F \frac{C_x}{N} = F(1 - \gamma)(1 - f_x), \quad \lambda_x > 1 \text{ supercritical.}$$

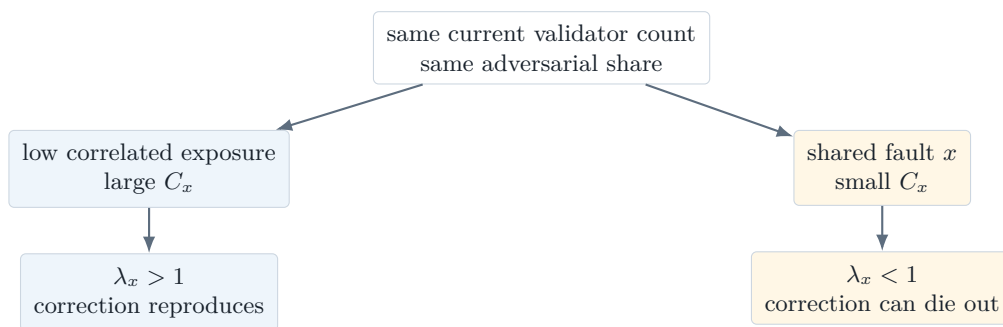
This substitution is a model extension, not a theorem attributed to the original ELVES analysis.

A minimal counterexample

Take the same validator count, the same adversarial share $\gamma = 0.20$, and the same illustrative escalation factor $F = 2$.

	shared-fault share f_x	independently correct share	λ_x
Population 1	0.10	$0.80 \times 0.90 = 0.72$	1.44
Population 2	0.50	$0.80 \times 0.50 = 0.40$	0.80

Both populations are 80% non-adversarial. Under this declared audit model, one is above the independent-correction threshold and the other below it. Ordinary validator counts and honest fractions therefore do not determine fault-specific corrective capacity.



The second question: what maintains correction over time?

Even if $\lambda_x > 1$ today, the ecosystem can lose the independent implementations, operators, effort, observability, or resources that supplied that margin. We therefore introduced the smallest explicit slow-state model we could use to separate *current correction* from *maintenance of future correction*.

Let C_t denote normalized independently corrective capacity, O_t maintenance observability/attribution, and R_t returned-resource capacity. The declared slow loop is $C \rightarrow O \rightarrow R \rightarrow C$. Coupling the next slow update back to the audit layer gives

$$\lambda_{x,t+1} = r_C \lambda_{x,t} + F k_{RC} R_t.$$

At current criticality, $\lambda_{x,t} = 1$, Lean verifies the exact one-step boundary

$$\lambda_{x,t+1} \geq 1 \iff F k_{RC} R_t \geq 1 - r_C.$$

Interpretation: return into corrective capacity must at least replace its attrition if a critical system is not to cross below the threshold on the next slow update.

Why a snapshot is not enough

The distinction does not depend on accepting the full three-variable maintenance model. In the reduced path $\lambda_t = m^t \lambda_0$, two systems can start at the same current value $\lambda_0 = 3/2$ but have opposite five-step status: with $m = 1$ the value remains $3/2$; with $m = 0.9$ it falls to approximately 0.886.

Present agreement $\neq$ present corrective capacity $\neq$ maintained corrective capacity $\neq$ future corrective resilience.

The full-state Lean result goes one step further. For the declared $C \rightarrow O \rightarrow R \rightarrow C$ dynamics, it identifies a canonical replacement condition and proves a coordinatewise forward-invariant region in which fast audit correction stays supercritical at every future cross-audit time. It also proves the complementary erosion mechanism: remove the immediate $R \rightarrow C$ return edge while $r_C < 1$, and positive corrective capacity declines at the next slow update.

What is proved, and what is not claimed

Machine-checked in Lean

- separation of current audit correction from slow maintenance viability;
- witnesses for all four cells of the two-threshold phase structure;
- the one-step bridge from slow state to next-audit reproduction;
- eventual erosion in the reduced model when $0 \leq m < 1$;
- the full-state forward-invariance persistence theorem;
- decline after deleting the immediate resource-return edge.

Deliberately not claimed

- that C_t, O_t, R_t are Gray-Paper state variables;
- that the coefficients are calibrated JAM quantities;
- that positive-systems, branching, or geometric-decay mathematics is novel;
- that the full slow system globally converges;
- that current JAM deployments have any particular correlated-fault rate.

Three questions for JAM/ELVES reviewers

1. **Coverage.** Is correlated honest implementation failure already covered by a JAM/ELVES assumption, verdict rule, sampling mechanism, or recovery mechanism that this analysis has missed?
2. **Security object.** If not, is *fault-specific independently corrective capacity* a meaningful quantity for reasoning about JAM execution auditing, or is there a better protocol-native object?
3. **Measurement.** If the distinction is meaningful, which observable testnet/deployment quantities would best estimate whether independent corrective capacity persists across audits—client/fault-family exposure, independent re-execution effort, operator diversity, attribution, resource flows, or something else?

The requested expert judgment is intentionally narrow: are we modelling a real residual failure mode of JAM, or has the protocol already closed this loop somewhere we have missed? If the latter, identifying that mechanism would directly falsify or substantially narrow the JAM-specific interpretation.

Where to inspect the full argument

The full derivation, claim ledger, Python experiments, and machine-checked Lean proofs are collected in [Distributed-Commons-Control, PR #22](#). The most relevant source paths are:

- `papers/correlated-honest-failure/` — manuscript and claim ledger;
- `formalization/JamRecurrentMaintenance.lean`;
- `formalization/JamResilienceDynamics.lean`;
- `formalization/JamMaintenancePersistence.lean`.